

Here is the same explanation in Markdown format:

```
# C++ Upcasting and Downcasting – ELI10
```

```
```cpp
class Base {};
class Derived : public Base {};
```

Imagine:

- Base = a general Animal
- Derived = a specific Dog

A Dog **is an** Animal .

So:

```
Derived* d = new Derived();
```

means:

```
d
|
v
+-----+
| Derived |
| (Dog object) |
+-----+
```

d is a pointer to a Derived object.

## 1. Upcasting: Derived\* to Base\*

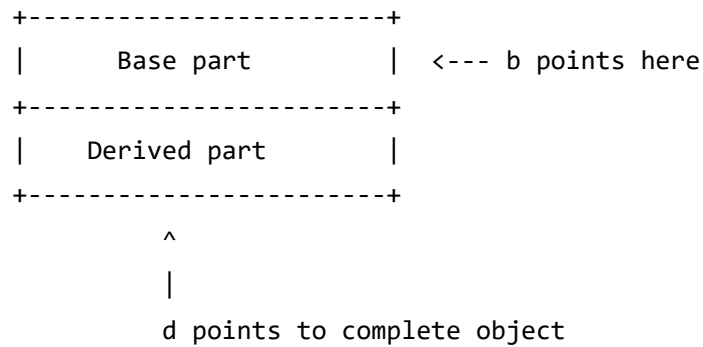
```
Base* b = static_cast<Base*>(d);
```

This is safe.

Why?

Because every `Derived` object contains a `Base` part.

Actual object in memory:



A `Derived` object is also a `Base` object.

Therefore:

```
Derived* ---> Base*
```

is called **upcasting**.

Usually, you don't even need `static_cast` :

```
Base* b = d; // Implicit upcast
```

This is enough.

## 2. Downcasting: `Base*` to `Derived*`

```
Derived* d2 = static_cast<Derived*>(b);
```

Now we are going in the opposite direction:

```
Base* ---> Derived*
```

This is called **downcasting**.

The important question is:

Does `b` really point to a `Derived` object?

In this example:

```
Derived* d = new Derived();
Base* b = d;
```

Yes!

The object is actually:

```
Derived object
|
+-- Base part
+-- Derived part
```

So this is safe:

```
Derived* d2 = static_cast<Derived*>(b);
```

Because `b` originally came from a `Derived` object.

## The Dangerous Case

Consider:

```
class Base {};
class Derived : public Base {};
class AnotherDerived : public Base {};
```

Now:

```
Base* b = new AnotherDerived();

Derived* d = static_cast<Derived*>(b);
```

What happened?

The actual object is:

Actual object:

```
+-----+
| AnotherDerived |
| |
| Base part |
+-----+
```

But you are telling the compiler:

"Trust me, b points to a Derived object."

The compiler says:

"Okay, I trust you."

But the truth is:

```
b ---> AnotherDerived object
 NOT a Derived object
```

Therefore:

```
Derived* d = static_cast<Derived*>(b);
```

is an **invalid downcast**.

The compiler does not perform a runtime check.

# Analogy

Imagine:

```
Base = Vehicle
Derived = Car
AnotherDerived = Bike
```

Every Car is a Vehicle .

So:

```
Car ---> Vehicle
```

is always safe.

You can say:

```
"This is a Vehicle."
```

because a car really is a vehicle.

But:

```
Vehicle ---> Car
```

is not always safe.

The vehicle could be:

```
Car
Bike
Bus
Truck
```

If you say:

```
Vehicle* v = new Bike();
```

```
Car* c = static_cast<Car*>(v);
```

you are telling the compiler:

```
"This vehicle is a car."
```

But it is actually a bike.

The compiler does not investigate. It simply trusts you.

# Complete Example

```
#include <iostream>

class Base
{
public:
 void baseFunction()
 {
 std::cout << "Base function\n";
 }
};

class Derived : public Base
{
public:
 void derivedFunction()
 {
 std::cout << "Derived function\n";
 }
};

int main()
{
 Derived* d = new Derived();

 // Upcasting
 Base* b = static_cast<Base*>(d);

 // Downcasting
 Derived* d2 = static_cast<Derived*>(b);

 d2->derivedFunction(); // Works

 delete d;
}
```

The flow is:

```

new Derived()
 |
 v
+-----+
| Derived |
+-----+
 ^
 |
 d : Derived*

 |
 | Upcast
 v

 b : Base*
 |
 | Downcast
 v

 d2 : Derived*

```

The original object was genuinely a `Derived` , so the downcast is valid.

## What `static_cast` Does NOT Do

This:

```
Derived* d2 = static_cast<Derived*>(b);
```

does **not** ask:

```
"Hey object, are you really a Derived ?"
```

It simply says:

```
"Compiler, treat this Base pointer as a Derived pointer."
```

So:

`static_cast`

means:

I, the programmer, promise that this conversion is valid.

## Safer Alternative: `dynamic_cast`

If the classes are polymorphic:

```
class Base
{
public:
 virtual ~Base() = default;
};

class Derived : public Base {};
class AnotherDerived : public Base {};
```

Then:

```
Base* b = new AnotherDerived();

Derived* d = dynamic_cast<Derived*>(b);
```

The runtime checks the actual object.

Since `b` points to `AnotherDerived` :

```
d == nullptr
```

So:

```
if (Derived* d = dynamic_cast<Derived*>(b))
{
 std::cout << "It is really Derived\n";
}
else
{
 std::cout << "It is NOT Derived\n";
}
```

# Simple Rule

Derived\* ---> Base\*

Always safe: **Upcasting**

Base\* ---> Derived\*

Potentially dangerous: **Downcasting**

# Comparison

Cast	Meaning	Safety
Derived* → Base*	Upcast	Safe
Base* → Derived* using static_cast	Downcast	Programmer must ensure safety
Base* → Derived* using dynamic_cast	Checked downcast	Runtime checked

# One-Line Memory Trick

`static_cast` says: "I know what I'm doing."

`dynamic_cast` says: "Let's check what the object really is."